

video_recording_and_analysis

Video Recording and Analysis Workflow

Last modified: 2026-06-20

Status: active

This document describes the video recording and OpenCV-based vision validation workflow for the Helix OTA project, as mandated by SS11.4.159(D).

Overview

Every recorded video **MUST** be machine-verified for genuine content. When no vision LLM is available, OpenCV (cv2) provides the fallback analysis engine.

The workflow: **RECORD -> EXTRACT -> VERIFY -> CHECK -> ACCEPT**

Requirements

Required

```
pip3 install opencv-python
```

Optional (for OCR text extraction)

```
pip3 install pytesseract
```

```
brew install tesseract
```

Recording videos

Record window-scoped MP4 per SS11.4.154:

Terminal window recording (via FFmpeg)

```
ffmpeg -f avcapturescreen -i "<window-id>" -t 30 \  
"$HOME/Downloads/helix_ota---<feature>---$(date -u +  
%Y%m%dT%H%M%S).mp4"
```

Recording filenames MUST follow the prefix convention per SS11.4.155:
<PREFIX>---<feature-or-scope>---<run-id>.<ext>

Default save path is \$HOME/Downloads per SS11.4.158(D).

Analyzing recordings with OpenCV

Analyze a single recording

```
python3 scripts/testing/analyze_recording.py ~/Downloads/  
helix_ota---server-health---20260618T235702Z.mp4
```

Output: summary table and per-recording JSON sidecar.

Batch analyze all recordings

```
bash scripts/testing/analyze_recording.sh ~/Downloads/
```

Output: aggregated JSON report at qa-results/opencv-analysis-
<timestamp>/REPORT.json.

Structured JSON output

```
python3 scripts/testing/analyze_recording.py ~/Downloads/  
helix_ota---server-groups---20260619T020010Z.mp4 --output  
report.json
```

What the analysis checks

Check	Method	Threshold
Freeze detection	SSIM between adjacent sampled frames	≥ 0.97 = frozen
Frame count	Total frames in video	≥ 2 for liveness
Flat/blank	Laplacian variance + contrast	< 15 = flat
Text regions	Sobel edge + morphology count	N regions found
OCR text	Tesseract on top text regions	confidence score

Understanding verdicts

Verdict	Meaning	Action
PASS - live content	Genuinely advancing content	Evidence valid
PASS - basic content check	Minimal frames, non-frozen	Manually verify
PASS (with freeze warnings)	Some stalling, content present	Review if concerning
FAIL - frozen	>60% near-identical frames	Re-record or investigate
FAIL - single-frame	Only 1 frame (screenshot)	Not a valid recording
FAIL - flat_blank	Very low contrast	Check capture device

Integration with HelixQA

HelixQA has a comprehensive OpenCV integration at submodules/helixqa/OPENCV_INTEGRATION_ARCHITECTURE.md with:

- Feature Detection and Element Matching (pkg/vision/element_detector.go)
- Text Detection and OCR Pipeline (pkg/vision/text_extractor.go)
- UI Layout Analysis (pkg/vision/layout_analyzer.go)

The Python-based analyzer (scripts/testing/analyze_recording.py) provides the immediate capability. For production HelixQA Challenge usage, integrate with the Go-based vision pipeline.

Anti-bluff notes

- OpenCV analysis MUST run on real recordings, never simulated
- Report real freeze/frame stats – honest results only
- When OpenCV analysis is insufficient (e.g. FLAG_SECURE displays), document as operator-attended per SS11.4.52
- A recording without content verification is NOT evidence per SS11.4.158